12、音频输出和音频录制

全志V3S嵌入式驱动开发（音频输出和音频录制）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2023-08-29 14:52:09 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量3.2k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 6

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数 1

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发 音视频

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S芯片的嵌入式驱动开发过程，涉及音频输出和录制。首先讲解了电路设计，包括麦克风和耳机接口。接着讨论设备树配置，确认声卡驱动已集成。然后介绍如何判断声卡是否正确加载，并通过编译安装TinyAlsa库。此外，还提供了寻找和播放wav音频文件的方法，以及录制和回放音频的操作步骤。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

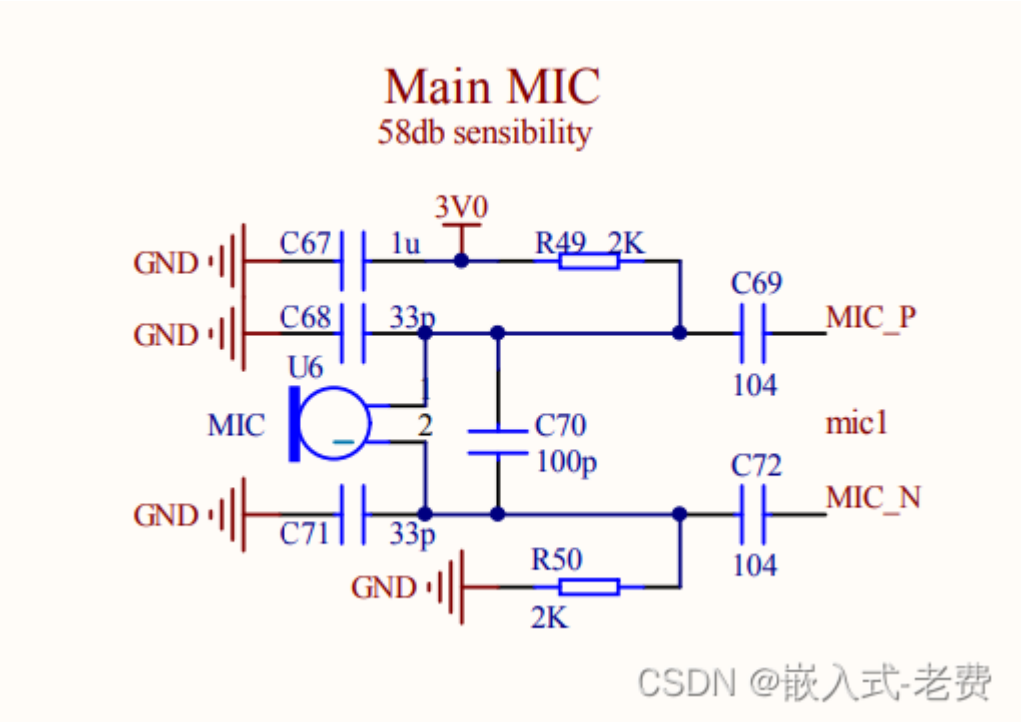
【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

之前在芯片公司的时候，基本没有看过音频这一块，只知道有个alsa框架这么个知识点。要驱动音频，需要两部分，一部分就是底层驱动，一部分就是alsa上层接口，两者缺一不可。 **对于荔枝派来说，底层的驱动其实都已经包含在linux kernel里面了，客户只要自己port好一个alsa库，或者类alsa库，就可以开始播放音频、录制音频了。**

1、电路

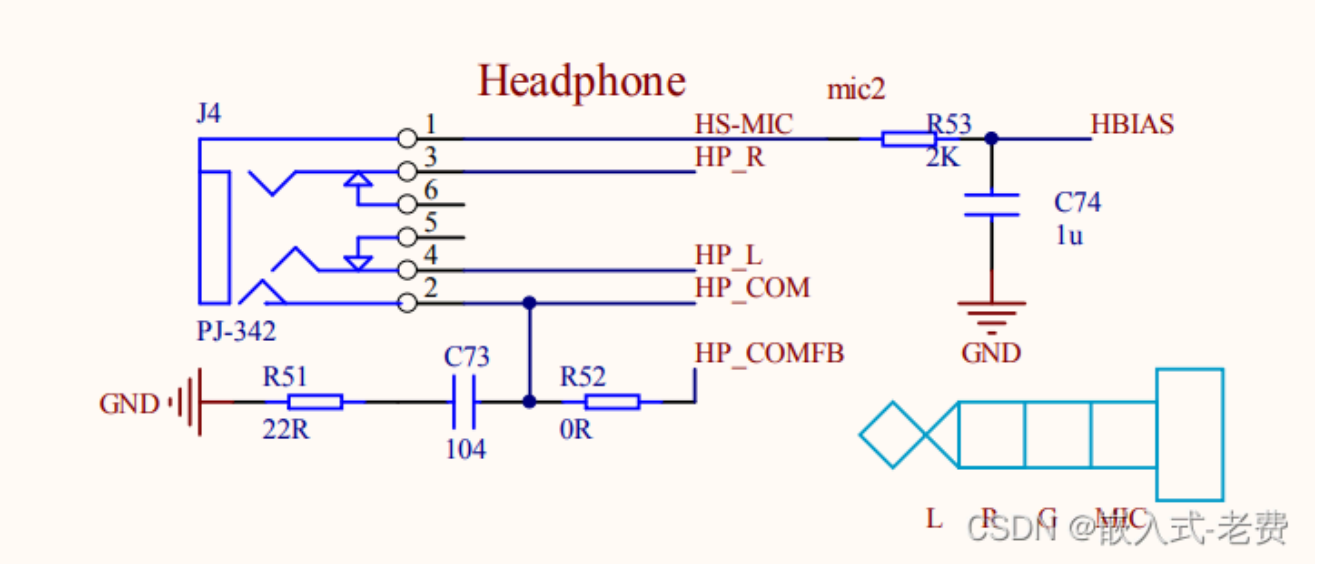
电路分成两个部分，一个是mic，也就是麦克风，录制音频用，

Open: 94581958d03d36494f7f32028f48b638_MD5.png



另外一个是headphone，也就是耳机，输出音频用，

Open: a8d056bd00e1e409613448d4c0f14cc0_MD5.png



不管是哪一个电路，通过观察发现，电路中并没有音频电路经常出现的iis接口。这就说明，v3s和网卡一样，本身已经集成了数模转换和功放功能了，不需要额外芯片了。遇到这种情况，一般soc厂商都会自己默默把驱动代码准备好，省着使用者去二次开发了。毕竟都是自己的东西，驱动写起来也得心应手，不用外人劳神劳力了。

2、设备树

本次使用的内核依然是linux-zero-4.14.y，顶层设备树是 sun8i-v3s-licheepi-zero-dock.dts。查看一下设备树的内容，可以发现声卡驱动已经集成到里面了，

```
&codec {
    allwinner,audio-routing =
        "Headphone", "HP",
        "Headphone", "HPCOM",
        "MIC1", "Mic",
        "Mic",  "HBIAS";
    status = "okay";
};
```

当然仅仅有这些还是不够的，进一步阅读sun8i-v3s.dtsi文件，可以看到codec的具体实现细节，

```
        codec: codec@01c22c00 {
            #sound-dai-cells = <0>;
            compatible = "allwinner,sun8i-v3s-codec";
            reg = <0x01c22c00 0x400>;
            interrupts = <GIC_SPI 29 IRQ_TYPE_LEVEL_HIGH>;
            clocks = <&ccu CLK_BUS_CODEC>, <&ccu CLK_AC_DIG>;
            clock-names = "apb", "codec";
            resets = <&ccu RST_BUS_CODEC>;
            dmas = <&dma 15>, <&dma 15>;
            dma-names = "rx", "tx";
            allwinner,codec-analog-controls = <&codec_analog>;
            status = "disabled";
        };

        codec_analog: codec-analog@01c23000 {
            compatible = "allwinner,sun8i-v3s-codec-analog";
            reg = <0x01c23000 0x4>;
        };
```

有了这两个配置，基本上声卡驱动就有保证了。好在这些都是默认配置，对于用户来说，就是什么都不需要做，系统上电后，声卡就已经加载好了。

3、判断声卡是否正确加载

判断声卡有没有加载好，主要有两个办法。第一，就是看内核启动日志；第二，就是看/dev/snd下面的节点有没有正确生成。 查看日志的话，如果启动ok，应该是可以看到这些打印的，

```
[    1.380188] sun4i-codec 1c22c00.codec: Codec <-> 1c22c00.codec
mapping ok
[    1.388822] sun6i-rtc 1c20400.rtc: setting system clock to 1970-01-
01 00:00:40 UTC (40)
[    1.397167] vcc5v0: disabling
[    1.400142] ALSA device list:
[    1.403106]   #0: V3s Audio Codec
```

不放心日志的话，可以输入ls -l /dev/snd进一步确认下，

```
# ls -l /dev/snd
total 0
crw-------    1 root     root      116,   0 Jan  1 00:00 controlC0
crw-------    1 root     root      116,  24 Jan  1 00:00 pcmC0D0c
crw-------    1 root     root      116,  16 Jan  1 00:00 pcmC0D0p
crw-------    1 root     root      116,  33 Jan  1 00:00 timer
```

另外，也可以查看下/sys/class/sound下面的节点，也可以算是一种方法，

```
# cd /sys/class/sound/
# ls
card0      controlC0  pcmC0D0c   pcmC0D0p   timer
```

不管是哪一种方法，基本上看到这些打印或者节点信息，就可以认为声卡被系统正确加载了。

4、编译安装TinyAlsa

前面我们说过，如果需要让声卡驱动起来，除了驱动本身之外，还需要一个alsa的库。TinyAlsa就是比较适合使用的那个库，下载位置在这，

```
https://github.com/tinyalsa/tinyalsa
```

下载好压缩包之后，拷贝到ubuntu虚拟机，直接用arm-linux-gnueabihf-交叉编译器来编译。解压一下，接着cd到解压目录后，直接输入这个命令，

```
 make CROSS_COMPILE=arm-linux-gnueabihf-
```

不出意外的话，在utils子目录下面，就可以看到四个编译好的工具，

```
tinycap
tinymix
tinypcminfo
tinyplay
```

通过观察编译日志，发现这四个程序都是静态链接libtinyalsa.a，所以直接拷贝四个文件即可，不需要拷贝其他动态库。

```
make -C src
make[1]: Entering directory '/home/feixiaoxing/Desktop/tinyalsa-
master/src'
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC    -c -o limits.o limits.c
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC    -c -o mixer.o mixer.c
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC    -c -o pcm.o pcm.c
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC    -c -o pcm_plugin.o pcm_plugin.c
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC    -c -o pcm_hw.o pcm_hw.c
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC    -c -o snd_card_plugin.o snd_card_plugin.c
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC    -c -o mixer_plugin.o mixer_plugin.c
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC    -c -o mixer_hw.o mixer_hw.c
arm-linux-gnueabihf-ar rv libtinyalsa.a limits.o mixer.o pcm.o
pcm_plugin.o pcm_hw.o snd_card_plugin.o mixer_plugin.o mixer_hw.o
arm-linux-gnueabihf-ar: creating libtinyalsa.a
a - limits.o
a - mixer.o
a - pcm.o
a - pcm_plugin.o
a - pcm_hw.o
a - snd_card_plugin.o
a - mixer_plugin.o
a - mixer_hw.o
arm-linux-gnueabihf-gcc  -shared -Wl,-soname,libtinyalsa.so.2 limits.o
mixer.o pcm.o pcm_plugin.o pcm_hw.o snd_card_plugin.o mixer_plugin.o
```

```
mixer_hw.o -o libtinyalsa.so.2.0.0
ln -sf libtinyalsa.so.2.0.0 libtinyalsa.so.2
ln -sf libtinyalsa.so.2 libtinyalsa.so
make[1]: Leaving directory '/home/feixiaoxing/Desktop/tinyalsa-
master/src'
make -C utils
make[1]: Entering directory '/home/feixiaoxing/Desktop/tinyalsa-
master/utils'
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC -O2   -c -o tinyplay.o tinyplay.c
arm-linux-gnueabihf-gcc -L ../src -pie  tinyplay.o ../src/libtinyalsa.a
-ldl -o tinyplay
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC -O2   -c -o tinycap.o tinycap.c
arm-linux-gnueabihf-gcc -L ../src -pie  tinycap.o ../src/libtinyalsa.a
-ldl -o tinycap
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC -O2   -c -o tinymix.o tinymix.c
arm-linux-gnueabihf-gcc -L ../src -pie  tinymix.o ../src/libtinyalsa.a
-ldl -o tinymix
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include -fPIC -O2   -c -o tinypcminfo.o tinypcminfo.c
arm-linux-gnueabihf-gcc -L ../src -pie  tinypcminfo.o
../src/libtinyalsa.a  -ldl -o tinypcminfo
make[1]: Leaving directory '/home/feixiaoxing/Desktop/tinyalsa-
master/utils'
make -C doxygen
make[1]: Entering directory '/home/feixiaoxing/Desktop/tinyalsa-
master/doxygen'
Makefile:11: "doxygen is not available please install it"
make[1]: Nothing to be done for 'all'.
make[1]: Leaving directory '/home/feixiaoxing/Desktop/tinyalsa-
master/doxygen'
make -C examples
make[1]: Entering directory '/home/feixiaoxing/Desktop/tinyalsa-
master/examples'
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include    pcm-readi.c ../src/libtinyalsa.so  -ldl -o pcm-readi
arm-linux-gnueabihf-gcc -Wall -Wextra -Werror -Wfatal-errors -I
../include    pcm-writei.c ../src/libtinyalsa.so  -ldl -o pcm-writei
```

```
make[1]: Leaving directory '/home/feixiaoxing/Desktop/tinyalsa-master/examples'
```

如何把四个文件拷贝到开发板，这个就不再赘述了。主要还是使用python http库和开发板上的wget命令来共同完成的。

5、寻找音频文件

目前，tinyalsa的工具只能播放wav文件，所以我们还需要找一个可以下载wav文件的网站。这里推荐一个网站链接给大家，

```
https://www.xmwav.com/
```

下载好音频文件之后，还是用同样的方法下载到开发板上面。

6、播放音频和录制音频

6.1 播放音频

播放音频的时候，除了正常插入耳机，还有三个地方需要注意下，不然听不到音频输出。第一，就是打开播放开关；第二，设置音量；第三，确认下前面的设置有没有对。这三个步骤都做完了，就可以播放音频了。设置命令主要是tinymix，播放命令是tinyplay。

```
# ./tinymix set 1 40
# ./tinymix set 2 1
# ./tinymix contents
Number of controls: 13
ctl type    num name                                    value
0   INT 1   DAC Playback Volume                     63 (range 0->63)
1   INT 1   Headphone Playback Volume               40 (range 0->63)
2   BOOL    2   Headphone Playback Switch                On, Off
3   INT 1   Mic1 Playback Volume                    3 (range 0->7)
4   INT 1   Mic1 Boost Volume                       4 (range 0->7)
5   INT 1   ADC Gain Capture Volume                 3 (range 0->7)
6   BOOL    2   DAC Playback Switch                     Off, Off
7   BOOL    2   DAC Reversed Playback Switch            Off, Off
8   BOOL    2   Mic1 Playback Switch                    Off, Off
9   BOOL    2   Mixer Capture Switch                    Off, Off
10  BOOL    2   Mixer Reversed Capture Switch           Off, Off
11  BOOL    2   Mic1 Capture Switch                     Off, Off
```

```
12  ENUM    2   Headphone Source Playback Route         > DAC, Mixer, ,
> DAC, Mixer,
```

播放刚才下载的音频，

```
# ./tinyplay  test.wav
playing 'test.wav': 2 ch, 44100 hz, 16-bit signed PCM
```

测试的时候，发现不是所有的音频都能放出来的，所以我们找了一个“挖呀挖”的wav文件，44100Hz，播放是没问题的，链接地址如下，

```
https://www.xmwav.com/mscdetail/133829.html
```

6.2 录制音频

录制音频和播放音频差不多。第一步打开录制开关，第二步确认配置ok。前两步都做好了之后，就可以用tinycap录制音频了，结束录制用ctrl+c，

```
# ./tinymix set 11 1
# ./tinymix contents
Number of controls: 13
ctl type    num name                                    value
0   INT 1   DAC Playback Volume                     63 (range 0->63)
1   INT 1   Headphone Playback Volume               40 (range 0->63)
2   BOOL    2   Headphone Playback Switch                On, Off
3   INT 1   Mic1 Playback Volume                    3 (range 0->7)
4   INT 1   Mic1 Boost Volume                       4 (range 0->7)
5   INT 1   ADC Gain Capture Volume                 3 (range 0->7)
6   BOOL    2   DAC Playback Switch                     Off, Off
7   BOOL    2   DAC Reversed Playback Switch            Off, Off
8   BOOL    2   Mic1 Playback Switch                    Off, Off
9   BOOL    2   Mixer Capture Switch                    Off, Off
10  BOOL    2   Mixer Reversed Capture Switch           Off, Off
11  BOOL    2   Mic1 Capture Switch                     On, Off
12  ENUM    2   Headphone Source Playback Route         > DAC, Mixer, ,
> DAC, Mixer,
# ./tinycap record.wav
Capturing sample: 2 ch, 48000 hz, 16 bit
```

如果想要确认录制的音频有没有问题，那么直接用tinyplay播放下即可，

```
# ./tinyplay record.wav
playing 'record.wav': 2 ch, 48000 hz, 16-bit signed PCM
Played 1601536 bytes. Remains 0 bytes.
```